

MCI_ADV 사용자 매뉴얼

대량 재난 사고 시뮬레이션 플랫폼

버전 1.0 | 2026년 4월

목차

1. 개요
2. 시스템 요구사항
3. 설치 방법
4. 프로젝트 구조
5. 빠른 시작 가이드
6. 시나리오 생성
7. 시뮬레이션 엔진
8. 대시보드 (Streamlit)
9. 배치 실험 파이프라인
10. 정책 조합 (64개 시나리오)
11. 평가 지표
12. API 설정
13. 문제 해결
14. 부록

1. 개요

MCI_ADV는 대량 재난 사고(Mass Casualty Incident) 발생 시 **환자 이송 최적화**를 위한 시뮬레이션 및 분석 플랫폼입니다. 구급차(AMB)와 드론(UAV)을 활용한 64가지 배차 정책 조합을 평가하여 주어진 사고 지점에서의 최적 전략을 도출합니다.

핵심 기능

- 시나리오 생성:** 카카오 모빌리티 API 기반 실시간/미래시간 교통정보 반영
- 시뮬레이션 엔진:** 이산 사건 시뮬레이션(DES), 64개 규칙 조합 (완전요인설계)
- 통계 분석:** ANOVA, Tukey HSD, Games-Howell 사후검정 자동 수행
- 웹 대시보드:** Streamlit 기반 인터랙티브 UI (지도, 분석, 데이터 편집)
- 배치 처리:** 수백~수천 개 좌표 대규모 실험

시스템 파이프라인

좌표 입력 → 시나리오 생성 → 시뮬레이션 (64규칙 x N샘플) → 분석 및 시각화

(위도, 경도) | 카카오 API 호출 | 이벤트 기반 시물 | ANOVA, 순위
CSV/YAML 출력 | results_*.txt | Folium 지도

2. 시스템 요구사항

하드웨어

- CPU: 멀티코어 권장 (배치 실험 시)
- RAM: 최소 8GB, 대규모 실험 시 16GB 권장
- 저장공간: 1000개 좌표 실험당 약 500MB

소프트웨어

- Python 3.9 이상
- Windows 10/11, macOS, 또는 Linux
- 인터넷 연결 (카카오 API 호출용)

필수 API 키

- **카카오 REST API 키** (아래 서비스 활성화 필요):
 - 카카오 모빌리티 (길찾기 API)
 - 카카오 로컬 (키워드/주소 검색)

3. 설치 방법

3.1 저장소 클론

```
git clone <repository-url>
cd MCI_ADV
```

3.2 의존성 패키지 설치

```
pip install -r requirements.txt
```

주요 패키지:

- **streamlit** (>=1.34) - 웹 대시보드
- **folium, streamlit-folium** - 지도 렌더링
- **pandas, numpy** - 데이터 처리
- **scipy, statsmodels** - 통계 분석
- **pyyaml** - 설정 파일
- **requests** - API 호출
- **haversine** - 직선 거리 계산
- **openpyxl** - 엑셀 파일 I/O
- **geopandas, shapely** - 지리 경계 처리 (배치 실험)

3.3 필수 데이터 파일

아래 파일을 **scenarios/** 디렉토리에 배치하세요:

파일	설명	필수
scenarios/안전센터와 소방서.csv	소방서/119안전센터 마스터 데이터	필수
scenarios/엑셀 결합 데이터.xlsx	병원 마스터 (병상, 수술실, 헬기장, 등급)	필수
scenarios/DISTANCE_MATRIX_FINAL.xlsx	사전 계산 거리 행렬	선택

4. 프로젝트 구조

```

MCI_ADV/
├── src/
│   ├── sce_src/
│   │   ├── orchestrator.py      # 시나리오 생성 모듈
│   │   ├── make_csv_yaml_dynamic.py # 마스터 오케스트레이터 (생성+실행 통합)
│   │   ├── BatchLab.py          # 시나리오 데이터 생성기 (API 호출, CSV/YAML)
│   │   └── BatchLab.py          # 레거시 배치 처리기
│   ├── sim_src/
│   │   ├── main.py              # 시뮬레이션 엔진
│   │   ├── SceneManager.py      # 진입점 (RunManager)
│   │   ├── EntityManager.py     # 설정 로드, 개체 초기화
│   │   ├── EventManager.py     # 개체 상태 관리 (환자/병원/구급차/UAV)
│   │   ├── RuleManager.py      # 이벤트 큐 및 시뮬레이션 루프
│   │   └── MCIEnvironment_gymnasium.py # 64개 정책 규칙 정의
│   │                                     # Gymnasium 환경 래퍼
│   └── vis_src/
│       ├── MCI_Streamlit.py     # 대시보드 및 시각화
│       └── pages/
│           ├── Generate.py      # 메인 대시보드 (3500+ 라인)
│           ├── ResultsCompare.py # 시나리오 생성 UI
│           └── BatchExperiment.py # 다좌표 결과 비교
│                                       # 배치 실험 대시보드
├── scenarios/                    # 생성된 시나리오 데이터
├── results/                      # 시뮬레이션 결과
├── experiment_logs/              # 실행 로그
├── experiment_1/                 # 배치 실험 파이프라인
└── requirements.txt

```

5. 빠른 시작 가이드

5.1 단일 좌표 (대시보드)

1. 대시보드 실행:

```
streamlit run src/vis_src/MCI_Streamlit.py
```

2. **Generate** 페이지로 이동 (사이드바)

3. 설정 입력:

- 프로젝트 경로 (자동 감지)
- 카카오 REST API 키
- 출발 날짜/시간
- 사고 좌표 (검색 또는 직접 입력)

4. 파라미터 설정:

- 사고 규모 (환자 수): 기본값 30
- 구급차 수: 기본값 30
- UAV 수: 기본값 3
- 구급차 속도: 40 km/h
- UAV 속도: 80 km/h
- 환자 인계 시간: 구급차 10분 / UAV 15분

5. "**Generate & Run**" 클릭 → 시나리오 생성 + 시뮬레이션 자동 실행

6. 결과 확인 (메인 대시보드):

- **Scenarios** 탭: 이벤트 로그, 환자 타임라인
- **Maps** 탭: Folium 지도 위 경로 시각화
- **Analytics** 탭: ANOVA, 시나리오 순위

5.2 명령줄 인터페이스 (단일 좌표)

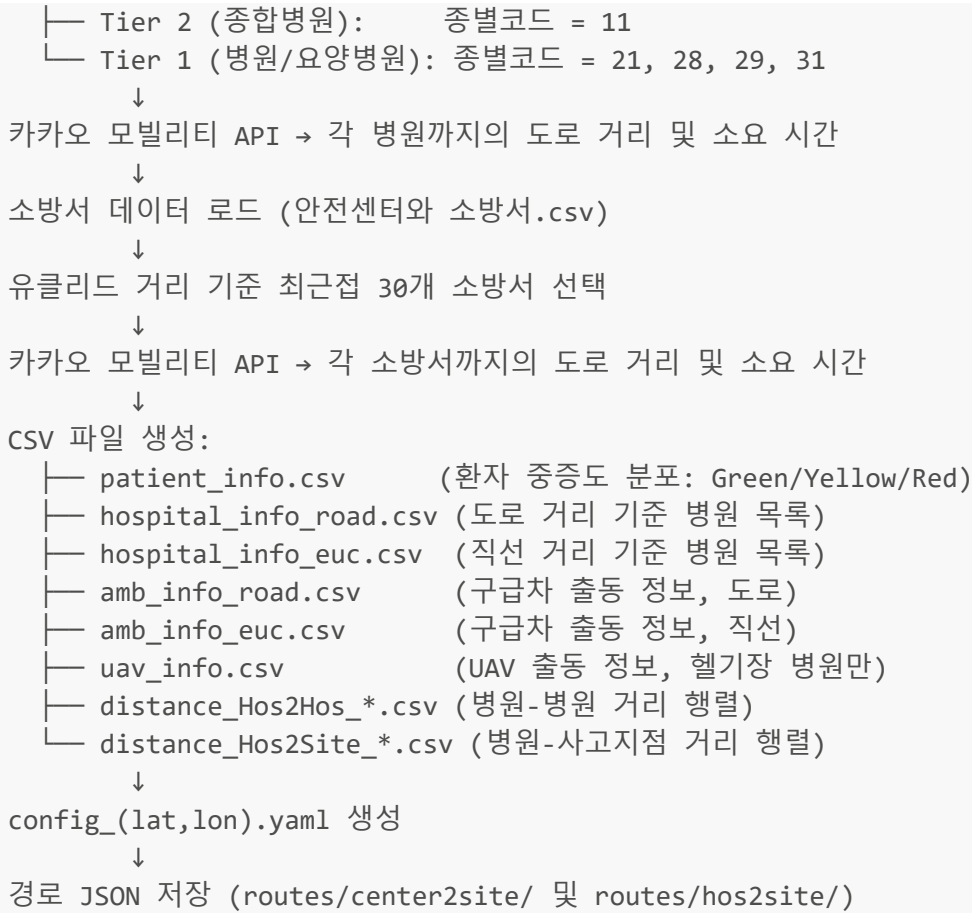
```
# 1단계: 시나리오 생성
python src/sce_src/make_csv_yaml_dynamic.py \
  --lat 37.5665 --lon 126.9780 \
  --kakao_api_key YOUR_API_KEY \
  --departure_time 202604031400 \
  --incident_size 30 --amb_count 30 --uav_count 3

# 2단계: 시뮬레이션 실행
python src/sim_src/main.py --config_path scenarios/exp_.../config_(lat,lon).yaml
```

6. 시나리오 생성

6.1 처리 흐름

```
입력 좌표 (위도, 경도)
  ↓
병원 마스터 데이터 로드 (엑셀 결합 데이터.xlsx)
  ↓
검색 반경 내 병원 필터링
  └─ Tier 3 (상급종합병원): 종별코드 = 1
```



6.2 주요 파라미터

파라미터	기본값	설명
<code>incident_size</code>	30	환자 수
<code>amb_count</code>	30	구급차 수
<code>uav_count</code>	3	UAV 수
<code>amb_velocity</code>	40	구급차 속도 (km/h)
<code>uav_velocity</code>	80	UAV 속도 (km/h)
<code>amb_handover_time</code>	10.0	구급차 환자 인계 시간 (분)
<code>uav_handover_time</code>	15.0	UAV 환자 인계 시간 (분)
<code>buffer_ratio</code>	1.5	병원 검색 반경 배율
<code>is_use_time</code>	True	API 기반 이동 시간 사용 여부 (미사용 시 거리/속도 계산)
<code>duration_coeff</code>	1.0	소요 시간 가중치 계수
<code>total_samples</code>	30	규칙당 시뮬레이션 반복 횟수
<code>random_seed</code>	0	재현성을 위한 랜덤 시드

6.3 환자 중증도 분포

환자는 아래 비율로 중증도가 무작위 배정됩니다:

중증도	비율	생존 모델	치료 가능 병원
Green (경증)	~60%	높은 생존율, 느린 감소	Tier 1, 2, 3
Yellow (중등증)	~20%	중간 생존율, 보통 감소	Tier 2, 3
Red (중증)	~20%	낮은 생존율, 급격 감소	Tier 3만 가능

6.4 병원 등급 체계

등급	한국 명칭	종별코드	수용 능력
Tier 3	상급종합병원	1	모든 중증도, 일부 헬기장 보유
Tier 2	종합병원	11	Green + Yellow만 치료
Tier 1	병원/요양병원	21, 28, 29, 31	Green만 치료

7. 시뮬레이션 엔진

7.1 아키텍처

이산 사건 시뮬레이션(DES) 방식으로, 우선순위 이벤트 큐를 사용합니다.

RunManager (main.py)

- ScenarioManager → 설정 로드, 개체 초기화
 - EntityManager → 환자/병원/구급차/UAV 상태 추적
 - EventManager → 이벤트 큐 관리 (heapq)
- RuleManager → 64개 규칙 조합 생성
- MCIEnvironment → Gymnasium 호환 환경 래퍼

7.2 이벤트 유형

이벤트	설명
onset	사고 발생, 환자 구조 필요
p_rescue	환자 구조 완료 (중증도 확인)
amb_arrival_site	구급차 사고 현장 도착
amb_arrival_hospital	구급차 병원 도착
uav_arrival_site	UAV 사고 현장 도착
uav_arrival_hospital	UAV 병원 도착
treat_start	병원에서 치료 시작
treat_end	치료 완료

7.3 시뮬레이션 루프

각 규칙(64개) x 각 샘플(N회 반복)에 대해:

1. `env.reset()` - 시나리오 초기화
2. `EventManager.run_next()`로 시간순 이벤트 처리
3. 규칙이 행동 선택: 어떤 환자를, 어떤 병원에, 어떤 이송수단으로
4. 경과 시간 기반 생존확률 계산
5. 지표 기록: Reward, Time, PDR, Reward_woG, PDR_woG

7.4 전원(Diversion) 규칙

병원 용량 초과 시 환자를 다른 병원으로 전원합니다:

- 구급차 환자: 환자 중증도에 맞는 가장 가까운 가용 병원으로 이송
- UAV + Red 환자: 헬기장이 있는 Tier 3 병원으로만 이송 가능

8. 대시보드 (Streamlit)

8.1 실행 방법

```
streamlit run src/vis_src/MCI_Streamlit.py
```

브라우저에서 <http://localhost:8501>로 자동 열립니다.

8.2 메인 페이지 탭

Settings (사이드바)

- 프로젝트 경로 선택
- 실험 ID 선택
- 좌표 선택
- 현재 좌표 위치를 보여주는 미니맵

Scenarios 탭

- 실험 로그 뷰어: 좌표별 로그 필터링
- 환자 타임라인: 구조시각, 이송수단, 병원, 도착시각, 치료완료 시각
- 이벤트 테이블: 전체 이벤트 로그 (Rule/Iteration 필터 가능)

Maps 탭

- 인터랙티브 Folium 지도: 경로 시각화
- 구급차 경로: 소방서→사고지점 (보라색), 사고지점→병원 (청록색)
- UAV 경로: 헬기장 병원→사고지점 (출동), 사고지점→병원 (이송)
- 교통 혼잡도 색상: 원활(녹색), 보통(파란색), 서행(노란색), 정체(빨간색)
- 경로 정보 팝업: 거리(km), 소요시간(분)

- 표시 경로 멀티셀렉트 (렌더링 성능을 위한 제한 가능)

Analytics 탭

- **RAW 결과 테이블:** 각 메트릭의 반복별 값
- **STAT 요약:** 64개 시나리오의 평균, 표준편차, 95% 신뢰구간
- **시나리오 순위:** Reward(내림차순), PDR(오름차순), Time(오름차순) 정렬
- **ANOVA 스위트:** 완전요인, 일원배치, RCBD 분석
- **사후검정:** Tukey HSD, Games-Howell
- **잔차 진단:** Shapiro-Wilk 정규성 검정, QQ plot, 히스토그램

Data Tables 탭

- 시나리오 CSV 파일 직접 편집
- 수정 전 자동 백업
- 수정된 데이터로 재실행 가능

Rerun 탭

- 기존 YAML 설정으로 시뮬레이션 재실행
- 시나리오 데이터 편집 후 유용

8.3 추가 페이지

Generate 페이지 ([pages/Generate.py](#))

- 전체 시나리오 생성 워크플로우
- 카카오 API 좌표 검색 (키워드 + 주소)
- 배치 좌표 입력 (다수 지점)
- 파라미터 프리셋 관리
- 생성 + 시뮬레이션 원클릭 실행

Results Compare ([pages/ResultsCompare.py](#))

- 다수 좌표 간 결과 비교
- 메트릭 나란히 비교

Batch Experiment ([pages/BatchExperiment.py](#))

- 5단계 워크플로우: 좌표 생성 → 확인 → 실행 → 진행 현황 → 시각화
- experiment_1 스크립트 함수 직접 import 재사용
- 중지/재개 기능이 있는 진행 추적

9. 배치 실험 파이프라인

9.1 개요

[experiment_1/](#) 디렉토리에는 다수 좌표에 대한 대규모 실험용 스크립트가 포함되어 있습니다.

9.2 단계별 가이드

1단계: 랜덤 좌표 생성

```
python experiment_1/generate_coords.py \  
  --n 1000 --seed 42 \  
  --shp experiment_1/ctprvn.shp \  
  --out experiment_1/coords_korea.csv
```

- 한국 육지 경계 내 N개 랜덤 좌표 생성
- Shapefile 기반 정밀 경계 판별
- `coord_id`, `lat`, `lon` 형식 CSV 및 미리보기 HTML 지도 출력

2단계: 배치 처리 실행

```
python experiment_1/batch_runner.py \  
  --coords experiment_1/coords_korea.csv \  
  --kakao-api-key YOUR_API_KEY \  
  --experiment-id exp_korea_random_1000 \  
  --departure-time 202603311400 \  
  --daily-limit 4900 \  
  --total-samples 30
```

주요 기능:

- **진행 추적:** `progress.json`에 각 좌표 처리 후 상태 저장
- **재개 기능:** 재시작 시 완료된 좌표 자동 건너뛰기
- **API 할당량 관리:** 일일 API 호출 추적, 한도 도달 시 일시 정지
- **오류 처리:** 실패 좌표 `--max-retries`로 재시도 가능

3단계: 결과 시각화

```
python experiment_1/visualize_coords.py \  
  --coords experiment_1/coords_korea.csv \  
  --progress experiment_1/progress.json \  
  --clip-pct 2 --outlier-n 5
```

출력물:

- `coords_map.html`: 색상으로 구분된 결과가 표시된 인터랙티브 Folium 지도
- `coords_map_hist.pdf`: 주요 지표 분포 히스토그램

9.3 Progress JSON 구조

```
{
  "experiment_id": "exp_korea_random_1000",
  "total": 1000,
  "statuses": {
    "0": {
      "status": "done",
      "sim_ok": true,
      "coord": "(lat,lon)",
      "api_calls": 45,
      "error": null
    }
  },
  "api_log": {
    "2026-03-31": 2450,
    "2026-04-01": 2400
  }
}
```

10. 정책 조합 (64개 시나리오)

시뮬레이션은 완전요인설계(Full Factorial Design)로 **64개 규칙 조합**을 평가합니다:

요인

요인	수준 수	설명
우선순위 (Phase)	2	START (초기 배차), ReSTART (재평가)
병원 선택	2	RedOnly (Red 환자 우선), YellowNearest (최근접 가용)
Red 행동	4	OnlyUAV, Both_UAVFirst, Both_AMBFirst, OnlyAMB
Yellow 행동	4	OnlyUAV, Both_UAVFirst, Both_AMBFirst, OnlyAMB

합계: $2 \times 2 \times 4 \times 4 = 64$ 개 조합

규칙 명명 규칙

```
{Phase}, {병원선택}, Red {Red행동}, Yellow {Yellow행동}
```

예: START, RedOnly, Red Both_UAVFirst, Yellow OnlyAMB

11. 평가 지표

지표	설명	방향
Reward	전체 환자의 생존확률 합	높을수록 좋음

지표	설명	방향
Time	전체 시뮬레이션 소요 시간 (분)	낮을수록 좋음
PDR	환자 사망률 (Patient Death Rate)	낮을수록 좋음
Reward w.o.G	Green 환자 제외 Reward	높을수록 좋음
PDR w.o.G	Green 환자 제외 PDR	낮을수록 좋음

생존확률 모델

각 환자의 생존확률은 시간 경과에 따라 중증도별로 감소합니다:

- **Green:** 느린 감소 (높은 기저 생존율)
- **Yellow:** 보통 감소
- **Red:** 급격한 감소 (시간에 민감한 중증 환자)

12. API 설정

12.1 카카오 REST API 키 발급

1. [카카오 개발자](#) 사이트 방문
2. 애플리케이션 생성
3. 서비스 활성화:
 - **카카오 모빌리티** (길찾기 API)
 - **카카오 로컬** (검색 API)
4. **REST API** 키 복사

12.2 사용되는 API 엔드포인트

엔드포인트	용도	좌표당 호출 수
모빌리티 길찾기	병원까지 도로 거리/소요시간	~20-50회
모빌리티 길찾기	소방서까지 도로 거리/소요시간	~30회
로컬 키워드 검색	좌표 검색 (대시보드 전용)	검색당 1회
로컬 주소 검색	주소 조회 (대시보드 전용)	검색당 1회

12.3 API 할당량 관리

- 카카오 무료 티어: **일 5,000회**
- 좌표당 약 **50-80회** API 호출 필요
- `batch_runner.py`가 `api_log`를 통해 일일 사용량 추적
- `--daily-limit` 설정 (기본값: 4900)으로 안전 마진 확보

13. 문제 해결

자주 발생하는 문제

문제	원인	해결 방법
시나리오 생성 <code>RuntimeError</code>	사고지점 자체에 도로 없음 (rc=102): 산, 바다, 무인도	버그가 아닌 정상 동작. 1000개 좌표 배치 실험 기준 223개가 이 유형. 소방서 경로 문제는 0건.
<code>Exception: Impossible to divert</code>	코드 로직 버그: UAV+Red 환자는 헬기장 +Tier3 병원에만 이송 가능. 유일한 해당 병원(예: 원광대, 용량16)이 가득 차면 비헬기장 Tier3 병원(전북대, 용량19)에 빈자리가 있어도 전원 실패.	알려진 엣지 케이스 (1000개 중 5개 영향). 병상 부족 아님 — 전체 Tier3 용량(35)이 환자 수(30)보다 충분. <code>diversion_rule()</code> 에 fallback 로직 추가 필요.
API 401 Unauthorized	잘못된 API 키	카카오 개발자 콘솔에서 키 확인
API 429 Rate Limit	일일 할당량 초과	24시간 대기 또는 할당량 증가
<code>UnicodeEncodeError: cp949</code>	Windows 콘솔 인코딩	<code>PYTHONIOENCODING=utf-8</code> 환경 변수 설정
YAML 키 순서 크래시	<code>yaml.dump</code> 의 <code>sort_keys=True</code>	<code>sort_keys=False</code> 사용 (수정 완료)
시뮬레이션 무한 대기	<code>stdout</code> 파이프 버퍼 가득 참	<code>Popen</code> + 데몬 스레드로 수정 완료

성능 팁

- 배치 실험: API 할당량 극대화를 위해 야간 실행 권장
- 대시보드: Analytics "Load Analysis Data" 버튼 사용 (지연 로딩)
- 지도: 표시 경로를 10-15개로 제한하면 렌더링 원활

부록

A. Config YAML 구조

```
entity_info:
  departure_time: "202604031400"
  patient:
    info_path: ./scenarios/exp_.../patient_info.csv
    total: 30
  hospital:
    info_road_path: ./scenarios/exp_.../hospital_info_road.csv
    info_euc_path: ./scenarios/exp_.../hospital_info_euc.csv
    distance_h2h_road_path: ./scenarios/exp_.../distance_Hos2Hos_road.csv
    distance_h2s_road_path: ./scenarios/exp_.../distance_Hos2Site_road.csv
  ambulance:
    info_road_path: ./scenarios/exp_.../amb_info_road.csv
    info_euc_path: ./scenarios/exp_.../amb_info_euc.csv
    velocity: 40
    handover_time: 10.0
  uav:
    info_path: ./scenarios/exp_.../uav_info.csv
    velocity: 80
```

```

handover_time: 15.0

output_path: ./results/exp_.../
totalSamples: 30
randomSeed: 0

```

중요: `entity_info`의 키 순서는 반드시 `departure_time` → `patient` → `hospital` → `ambulance` → `uav`여야 합니다. 이 순서가 바뀌면 `ScenarioManager`가 dict 삽입 순서로 개체를 처리하기 때문에 크래시가 발생합니다.

B. 결과 파일 형식

RAW (`results_(lat,lon).txt`):

```

START, RedOnly, Red OnlyUAV, Yellow OnlyUAV 12.5 13.2 11.8 ...
START, RedOnly, Red OnlyUAV, Yellow Both_UAVFirst 14.1 12.9 ...
...

```

각 줄: 규칙 라벨 + N개 샘플 값. 5개 메트릭 블록(Reward, Time, PDR, Reward_woG, PDR_woG), 각각 64줄.

STAT (`results_(lat,lon)_stat.txt`):

```

START, RedOnly, Red OnlyUAV, Yellow OnlyUAV 12.50 0.72 0.26

```

각 줄: 규칙 라벨 + 평균, 표준편차, 95% 신뢰구간 반폭.

C. 소방서 데이터 형식

안전센터와 소방서.csv (CP949 인코딩):

```

기관명,주소,전화번호,x좌표,y좌표,관할구역,대수
강남소방서,서울특별시 강남구...,02-3400-1119,127.0495,37.4979,...,5

```

D. 병원 엑셀 형식

엑셀 결합 데이터.xlsx:

- 주요 컬럼: `요양기관명`, `종별코드`, `수술실수`, `병상수`, `헬기장 여부`, `x좌표`, `y좌표`, `주소`, `전화번호`

MCI_ADV - 대량 재난 사고 시뮬레이션 플랫폼
문의 사항은 GitHub 저장소를 방문해 주세요.